

VisionInspect AI Milestone 4

VISIONINSPECT AI

AI/ML MODULE | MILESTONE 4

System Integration, Portable Deployment, and Operational Delivery

End-to-end inspection orchestration, persistence, review and rework, reporting, analytics, security, deployment configuration, and system handover

Submission field	Details
Project	VisionInspect AI
Assigned module	Member 4 - AI/ML, integrated with the full platform
Prepared by	Himanshu Sekhar Parhi
Mentor	Sai Jyoteesh
Programme	Infosys Springboard Programme
Document version	Milestone 4 Submission - Version 1.0
Submission date	15 August 2026

Document Control

This report records Milestone 4 of VisionInspect AI. It explains how the AI/ML outputs developed in Milestones 1 to 3 are connected to the FastAPI backend, MongoDB records, image storage, Next.js user interface, reviewer workflow, rework tickets, PDF reports, analytics, audit history, and portable deployment configuration.

Submission sequence: Milestone 1 - 25 July 2026; Milestone 2 - 1 August 2026; Milestone 3 - 8 August 2026; Milestone 4 - 15 August 2026. This report completes the weekly progression from data and detection to an operational inspection platform.

Milestone status: Completed for the project scope. The application exposes the full inspection workflow, all 15 category runtimes are portable, the backend and frontend build successfully, and 50 automated tests pass.

Control item	Recorded value
Milestone	4 - System Integration, Portable Deployment, and Operational Delivery
Primary notebooks	08_multicategory_end_to_end_inference.ipynb; 10_artifacts_testing_and_integration.ipynb
Application layers	Next.js frontend; FastAPI backend; MongoDB; local/Cloudinary media storage
Runtime coverage	15 MVTec categories; portable baseline and defect classifier artifacts
Inspection modes	Manual image, batch up to 20 images, and camera simulation
Operational outputs	History, manual review, rework, PDF report, analytics, audit logs
Software checks	50 Python tests passed; Next.js production build passed
Repository branch	Himanshu-parhi-ai/ml

Milestone Snapshot

Value	Meaning
15 / 15	Product categories with runnable compact anomaly artifacts
15 / 15	Categories with subtype classifier artifacts
30	Images exercised in the two-image-per-category integration smoke run
0	Runtime errors in that smoke run
20	Maximum images accepted by one batch request
8	MongoDB document types registered by the application
50	Automated Python tests passed
13 / 13	Static pages generated by the frontend production build

Table of Contents

Section	Page
1. Executive Summary	3
2. Milestone Scope and Acceptance Criteria	4
3. Integrated System Architecture	5
4. ML-to-Backend Runtime Contract	6
5. Image Acquisition and Inspection Orchestration	8
6. Persistence, Media Storage, and Traceability	9
7. Frontend Inspection and Review Experience	9
8. Batch Inspection and Camera Simulation	10
9. Rework, Reports, Analytics, and Audit History	11
10. Authentication, Roles, and Security Controls	12
11. Portable Runtime and Deployment Configuration	13
12. Practical End-to-End Demonstration	14
13. Software Testing and Build Results	16
14. Operational Limits and Mitigations	17
15. Milestone Completion and Project Handover	17
References	18
Appendix A. API Inventory	18
Appendix B. Persistence and Configuration Map	19
Appendix C. Deliverable and Evidence Index	19

List of Figures and Key Terms

Figure	Description
1	Integrated VisionInspect AI system architecture
2	ML-to-application output contract
3	Single-image inspection request sequence
4	Human review and rework flow
5	Portable deployment topology
6	All-category portable inference smoke run
7	Executed anomaly map and heatmap
8	Executed backend-ready result
9	Full Python test result
10	Frontend production build result

Key terms

Term	Meaning in VisionInspect AI
Inspection contract	Stable set of values and images returned by the ML runtime to the backend.
Operational record	MongoDB inspection document that links evidence, metadata, decision, and review state.
Portable runtime	Committed compact artifacts that run without a large Anomalib checkpoint.
Camera simulation	Repeatable inspection of bundled or local MVTec sample frames; not a physical PLC camera.
Rework ticket	Tracked action item created when a reviewer sends an inspection for repair.
Audit event	Actor, action, entity, metadata, and timestamp retained for traceability.

1. Executive Summary

Milestone 4 turns the model pipeline into a usable quality-inspection system. A user signs in, selects a product category, uploads an image or starts a simulated camera frame, and supplies production context. The backend validates the request, executes the category-specific ML pipeline, stores visual evidence, saves a complete inspection record, and returns an interpretable result to the frontend.

The result contains the Good/Defective prediction, anomaly score, separate detection and subtype confidence values, defect type, defect area, heatmap, severity components, severity level, Pass/Review/Fail decision, recommendation, active runtime, and fallback status. One inspection identifier links this evidence with the uploader, product, batch, line, shift, reviewer notes, rework ticket, generated report, analytics, and audit events.

The clone-and-run path uses compact category profiles, a shared ResNet18 ONNX feature extractor, and category-specific classifiers. Large PaDiM or PatchCore checkpoints remain optional. This design allows the website to work without the complete MVTec dataset while keeping a clear upgrade path for a compute-capable server.

Verified result: The all-category notebook processed 30 real MVTec images across 15 categories with 0 runtime errors. The full project test suite passed 50 tests, and the Next.js production build completed successfully.

1.1 Delivered outcome

- A stable end-to-end contract connects model output to the backend and frontend.

- Manual, batch, and simulated-camera inspection share the same inference and persistence path.
- Review, rework, reporting, analytics, and audit history use saved inspection data.
- JWT authentication, account approval, and role-based page/action controls are implemented.
- Local-first storage and compact model artifacts support portable execution after cloning.
- Deployment definitions are present for a Vercel frontend and Render backend.

2. Milestone Scope and Acceptance Criteria

2.1 Objective

The objective is to connect the complete AI/ML pipeline to an operational manufacturing workflow and make the project understandable, traceable, testable, and portable for demonstration and handover.

2.2 Included work

- FastAPI orchestration for image validation, category selection, model execution, storage, and response serialization.
- MongoDB documents for users, inspections, reports, rework, products, lines, batches, and audit logs.
- Next.js pages for login, upload, camera simulation, history, rework, reports, analytics, model metrics, and users.
- Manual, batch, and camera-simulation input workflows.
- Reviewer actions, rework ticket lifecycle, PDF report generation, and production analytics.
- Portable runtime selection, model health reporting, environment configuration, and cloud deployment definitions.
- Full automated test run and frontend production build check.

2.3 Scope boundaries

- Camera acquisition is simulated with stored frames; direct industrial camera, PLC, and SCADA control is outside this submission.

- Anomaly localization is implemented through anomaly maps and heatmaps; there is no separate YOLO object detector.
- MVTec data demonstrates the workflow but is not redistributed with the repository.
- The system provides engineering quality recommendations; factory release policy remains a human responsibility.
- Advanced checkpoints are optional because their size and compute requirements exceed the portable default path.

2.4 Acceptance criteria

Criterion	Completion condition	Status
Authenticated workflow	Protected pages require a valid user token	Met
Inspection orchestration	Valid image and category produce a persisted structured result	Met
Input modes	Manual, batch, and camera simulation use the same result contract	Met
Traceability	Inspection includes product/batch/line/shift/operator/source and timestamps	Met
Human control	Authorized reviewer can approve, reject, or create a rework ticket	Met
Reporting	A two-page PDF can be generated from a stored inspection	Met
Analytics	Dashboard summary and CSV export use inspection records and filters	Met
Portability	All 15 categories expose runnable compact artifacts	Met
Software checks	Python tests and frontend production build complete	Met

3. Integrated System Architecture

VisionInspect AI separates responsibilities by layer. The frontend manages user interaction. FastAPI owns security, request validation, orchestration, persistence, and operational APIs. The ML package owns image analysis and returns evidence. MongoDB and media storage retain the quality record. This separation allows each layer to change without duplicating model logic.

VisionInspect AI integrated system architecture

The ML runtime is one service boundary inside a traceable quality workflow.

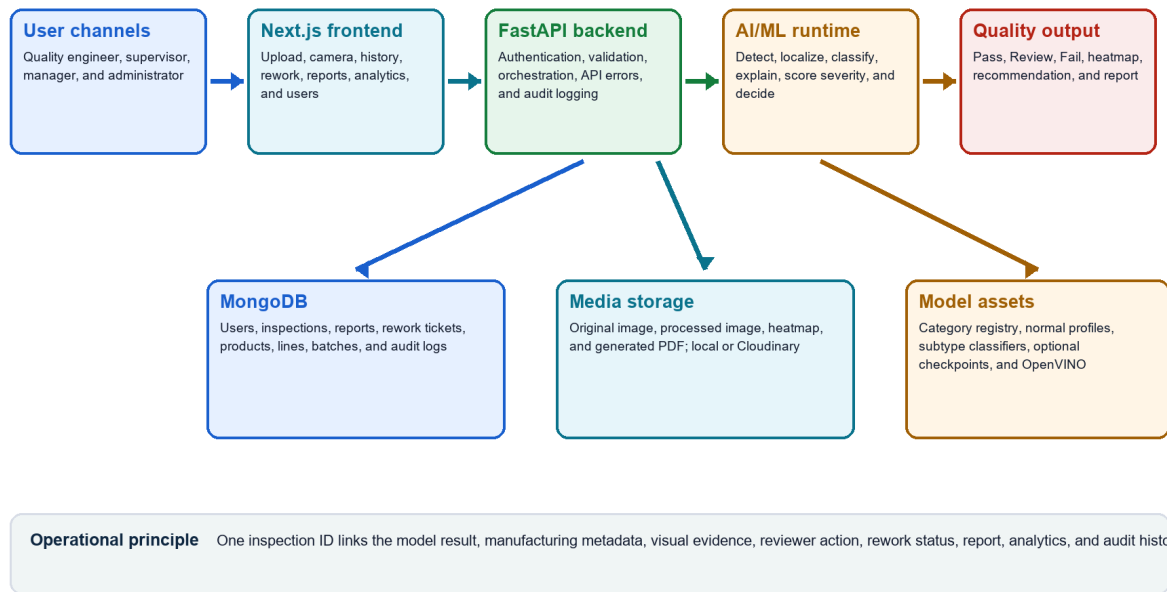


Figure 1. Integrated VisionInspect AI system architecture

Source: Generated from the implemented frontend, FastAPI routes and services, ML runtime, MongoDB documents, and storage service.

3.1 Layer responsibilities

Layer	Primary responsibility
Frontend	Collect input, call APIs, show progress/errors, visualize result, support human actions
Backend	Authenticate, validate, orchestrate ML/storage, persist records, expose reports and analytics
AI/ML	Preprocess, detect anomaly, classify subtype, localize, explain, score severity, decide
MongoDB	Retain users, production context, inspections, review/rework, reports, analytics source, audit events
Media storage	Publish original/processed images, heatmaps, and generated report files
Deployment	Build frontend, run backend, inject secrets/configuration, expose health checks

3.2 Design decision

Single source of model logic: The frontend does not reproduce anomaly thresholds or severity calculations. It displays fields returned by the backend. The backend calls the shared `ml.inference.inspect_image` function, which keeps model behavior consistent across notebooks, CLI use, tests, and the website.

4. ML-to-Backend Runtime Contract

The backend supplies an image path, normalized product category, optional critical zones, and active threshold settings. The ML runtime returns arrays for visual storage and JSON-safe values for the inspection document. The backend removes large arrays from the API payload after saving the processed image and heatmap.

ML-to-application output contract

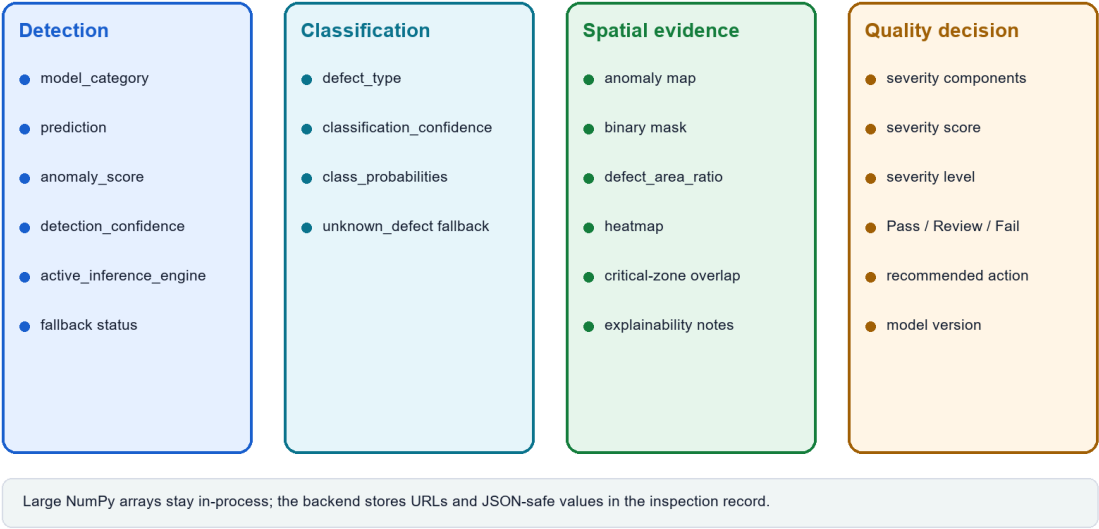


Figure 2. ML-to-application output contract
Source: Generated from ``ml/inference.py``, ``ml/predict.py``, ``backend/app/services/prediction_service.py``, and the inspection model.

Contract group	Fields
Input	image path; category; critical zones; detector/classifier paths; runtime thresholds
Binary detection	prediction; anomaly score; detection confidence; active engine; fallback state
Subtype	defect type; subtype confidence; class probabilities; classifier error when present
Localization	anomaly map; predicted mask; defect area; centroid; regions; critical overlap; heatmap
Quality	severity components; score; level; Pass/Review/Fail; recommendation
Operations	model version; processed image; explainability notes; runtime timing

4.1 Confidence handling

Detection confidence describes confidence in Good versus Defective and is the confidence term used in severity. Classification confidence describes confidence in the subtype label. If subtype evidence is unavailable or conflicts with a detected anomaly, the runtime returns ``unknown_defect`` instead of inventing a category.

5. Image Acquisition and Inspection Orchestration

Single-image inspection request sequence

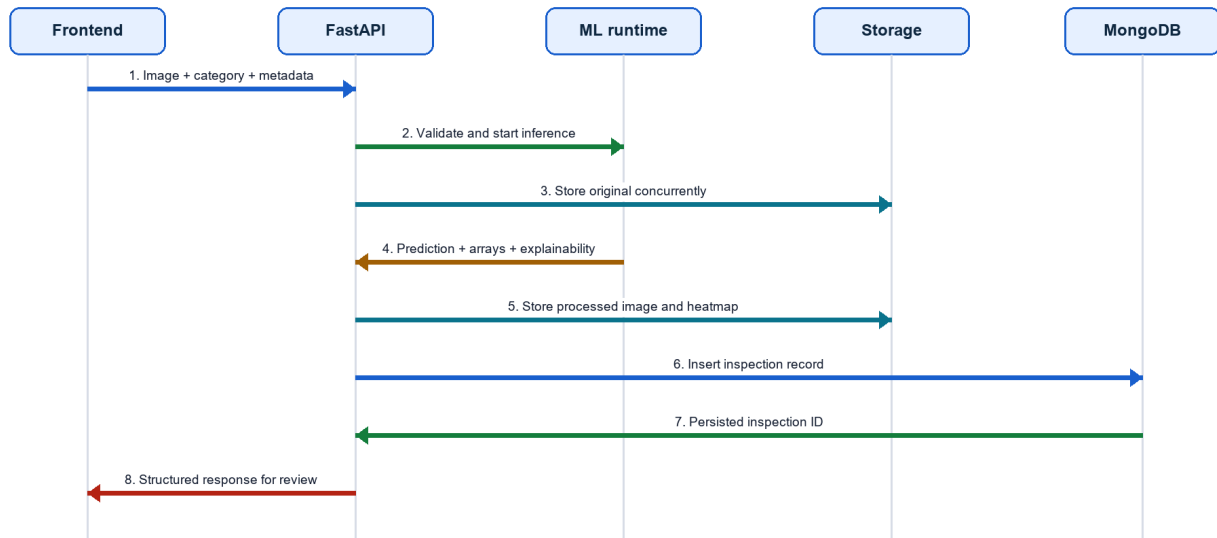


Figure 3. Single-image inspection request sequence

Source: Generated from the `/inspections/inspect` route and prediction/storage services.`

5.1 Request processing

Step	Processing action
1	The frontend sends one image, the selected category, and production metadata as multipart form data.
2	The backend checks file extension, non-empty content, maximum size, and whether OpenCV can decode the image.
3	Blank metadata is filled with traceable defaults without overwriting user-entered values.
4	Original-image storage and ML inference start concurrently to reduce request time.
5	The category registry resolves the normal profile, classifier, metadata, optional checkpoint, and optional OpenVINO export.
6	Inference returns detection, classification, localization, severity, QA decision, explainability, and visual arrays.
7	Processed image and heatmap are saved; local URLs or Cloudinary URLs are returned.
8	One inspection document is inserted, an audit event is recorded, and the serialized response is sent to the frontend.

5.2 Input validation and errors

Condition	Status	Behavior
Unsupported extension	400	PNG, JPG/JPEG, BMP, TIFF, and WebP are accepted
Empty or undecodable file	400	Request stops before model execution
File above configured limit	413	Default maximum is 10 MB
Missing category artifacts	500	Actionable runtime-artifact message is returned
Online storage timeout	503	Production message asks the user to retry
Frontend request timeout	408	User sees a clear long-running request message

6. Persistence, Media Storage, and Traceability

MongoDB is the operational source of truth. Media files remain outside the database and are referenced by URL and local path. This prevents large binary arrays from expanding inspection documents while preserving the evidence needed by the frontend and PDF reports.

MongoDB document	Stored purpose
users	Identity, role, approval status, active state, login and approval timestamps
inspections	Images, prediction, classification, severity, metadata, review and rework links
reports	Inspection link, report URL/path, type, and generation time
rework_tickets	Ticket number, assignment, priority, due date, status, and resolution
products	Product ID, category, critical zones, and active state
production_lines	Line ID, name, location, and active state
batch_records	Batch, product, line, shift, and status
audit_logs	Actor, role, action, entity, metadata, and timestamp

6.1 Media storage

- Development default: files are kept under backend uploads and served through ``/uploads``.
- Online option: Cloudinary stores original, processed, and heatmap assets when explicitly enabled.
- Report downloads are authenticated and regenerated from the inspection if the local PDF is missing.
- Partial failures trigger best-effort rollback of local and Cloudinary assets.
- Inspection and batch records do not expire automatically; historical QA data is retained.

6.2 Traceability chain

Stage	Trace fields
Acquisition	uploader, source type, source label, original URL
Production	category, product, batch, line, shift, operator
AI	model, engine, scores, class probabilities, heatmap, explanation
Quality	decision, recommendation, reviewer, notes, review time
Corrective action	rework ticket, assignee, due date, priority, resolution
History	created/updated times plus audit events

7. Frontend Inspection and Review Experience

The Next.js interface presents the application as a working quality console rather than a notebook viewer. ``AppShell`` verifies the stored token, retrieves the current user, redirects unauthenticated users to login, and blocks pages outside the user's role.

Page	Operational purpose
Login	Sign in or submit a role-based registration request
Dashboard	Quality KPIs and recent inspection activity
Upload	Manual image, metadata, batch input, result, heatmap, and report action
Camera	Select category/defect stream and inspect sequential simulated frames
Inspection History	Filter records, view evidence, update metadata, and review result
Rework Queue	Track ticket priority, assignee, due date, status, age, and resolution
Analytics	Filter and visualize quality trends, severity, defect type, and line performance
Reports	List and download authenticated inspection reports
Model Metrics	View artifact status, category metrics, and adjust runtime thresholds
Users	Approve/reject registrations and manage role-appropriate accounts

7.1 Inspection result presentation

- Original image and generated heatmap are shown together.
- Prediction, defect type, two confidence values, anomaly score, severity, and decision are separated clearly.
- Explainability states the threshold, defect area, heatmap intensity, critical-zone result, and reason for Pass/Review/Fail.
- Slow requests, server timeouts, validation failures, request IDs, and network failures are presented with actionable messages.
- Review controls appear only to authorized roles and require notes for rework.

7.2 Workflow states

The upload page moves through Image selected, Context added, Completed, and Ready states. These labels describe progress rather than acting as inactive navigation buttons. Loading indicators and disabled actions prevent accidental duplicate inspection or report requests.

8. Batch Inspection and Camera Simulation

8.1 Batch inspection

A batch request accepts one to twenty images. Every image is processed through the same single-inspection function, which guarantees the same validation, category runtime, metadata completion, persistence, storage, and audit behavior. Individual failures are returned beside successful records rather than cancelling the whole batch.

Batch summary field	Meaning
Requested	Number of files submitted
Succeeded / failed	Files that produced records versus per-file errors
Good / Defective	Binary anomaly outcomes
Pass / Review / Fail	Operational QA decisions
Critical	Inspections at the highest severity level
Average confidence	Mean detection confidence across successful inspections

8.2 Camera simulation

Camera simulation reads real category test images when the dataset is available and falls back to bundled bottle frames when it is not. The user chooses a category and optional defect stream, then the frontend requests sequential frames. Each frame becomes an ordinary persisted inspection with source type `camera\_simulation`.

Scope statement: Camera simulation proves the acquisition and inspection loop in a repeatable way. It is not a physical industrial camera driver and does not claim PLC timing, trigger, or line-control integration.

9. Rework, Reports, Analytics, and Audit History

9.1 Human review and rework

Human review, rework, and release flow

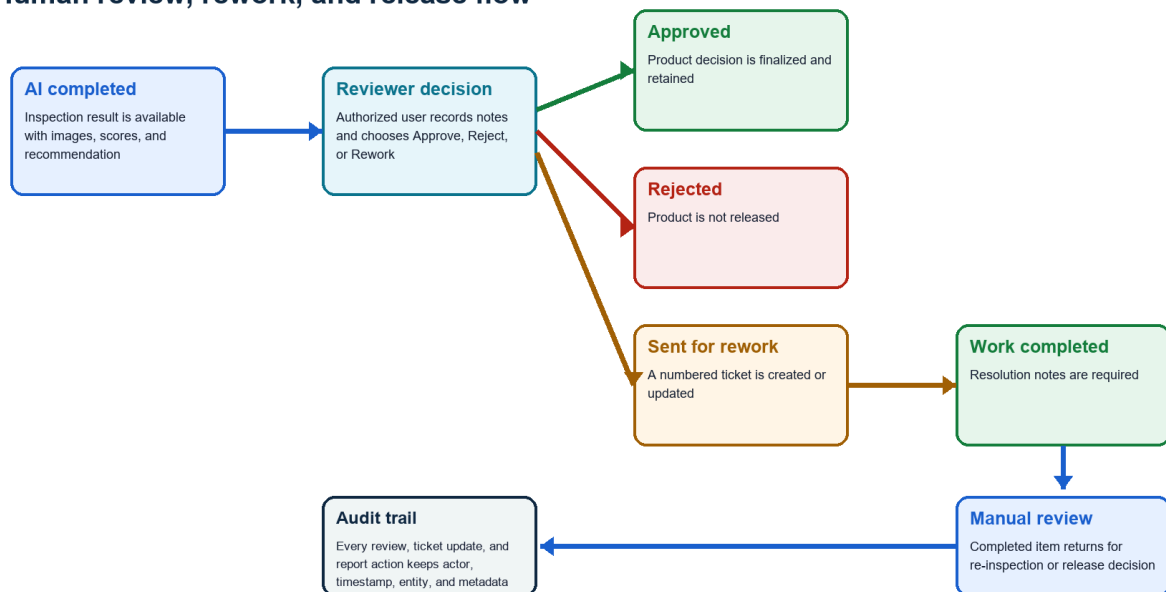


Figure 4. Human review, rework, and release flow

Source: Generated from inspection and rework routes, document models, and role controls.

Sending an item to rework does more than change a label. The backend creates or updates a numbered ticket, copies product and defect context, calculates priority from severity, records assignment and due date, and links the ticket back to the inspection. Completing or closing a ticket requires resolution notes and returns the inspection to manual review.

9.2 PDF reports

ReportLab generates a two-page A4 report. Page 1 contains decision, prediction, defect type, confidence, model version, severity, production metadata, original image, and heatmap. Page 2 contains severity components, action, quality summary, explainability, reviewer information, and review notes.

9.3 Analytics

Analytics group	Returned information
KPI	Total, Good, Defective, Pass, Review, Fail, defect rate, confidence, severity, critical count
Distributions	Defect type, severity, review status, line, shift, and source type
Trends	Daily totals, rates, defect types, and severity levels
Line performance	Pass/review/fail rate, defect rate, average severity
Filters	Date range, product, production line, and defect type
Export	Authenticated CSV of inspection records

9.4 Audit history

Login, registration, approval, inspection completion, camera actions, metadata changes, reviewer actions, rework ticket changes, report generation, user management, and production-catalog changes are recorded as audit events.

10. Authentication, Roles, and Security Controls

10.1 Account lifecycle

Step	Account action
1	A new user provides name, email, password, and requested role.
2	The account is stored as pending and inactive.
3	An authorized manager reviews and approves or rejects the request.
4	Approved users receive a JWT after password verification.
5	Protected APIs decode the token, load the active user, and check required roles.

10.2 Role matrix

Role	Main application access
Quality engineer	Dashboard, upload, camera, own/history inspections, reports, permitted review actions
Factory supervisor	Engineer access plus rework queue, analytics, and managed engineer accounts
Quality manager	Supervisor access plus model metrics/settings and broader user management
Administrator	All application pages and account roles

10.3 Security behavior

- Passwords are bcrypt hashes; plain passwords are not stored.
- JWTs use expiry and a configurable secret key.

- CORS origins are configuration-driven.
- Responses include request identifiers and security headers such as `X-Content-Type-Options: nosniff`.
- Users cannot view other users' inspections unless their role permits wider access.
- Production secrets belong in hosting environment variables and are excluded from source control.

11. Portable Runtime and Deployment Configuration

Portable deployment topology

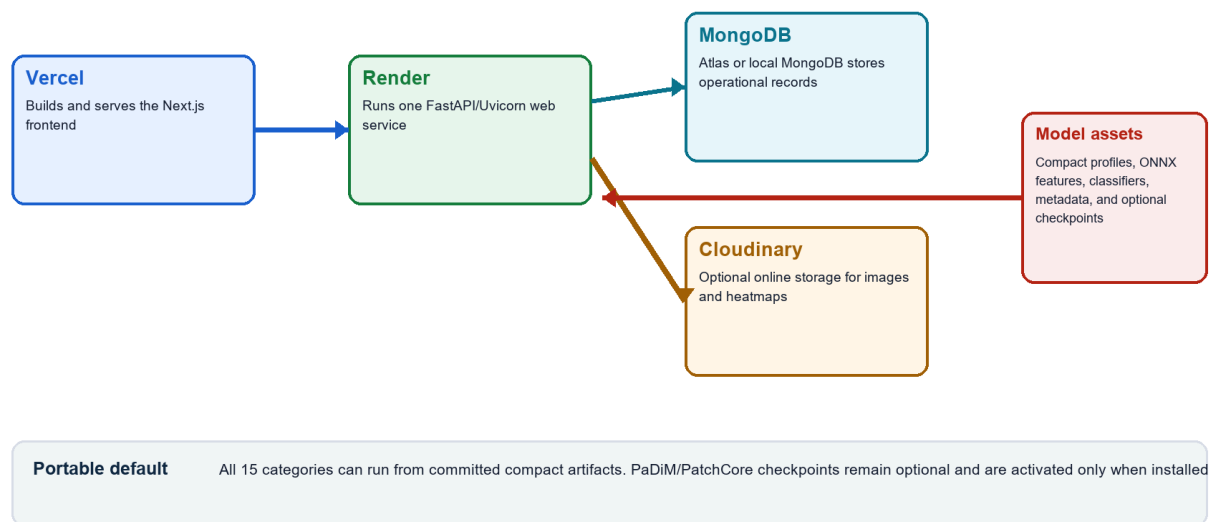


Figure 5. Portable deployment topology

Source: Generated from ``.env.example``, `deployment/render.yaml``, `frontend/vercel.json``, backend settings, and the model registry.

11.1 Runtime selection

Mode	Availability	Coverage	Purpose
Portable hybrid	Included	All 15 categories	Normal memory + OpenCV spatial evidence + subtype classifier
OpenVINO	Conditional	Categories with exported XML/BIN	CPU-optimized advanced inference
PaDiM/PatchCore	Conditional	Category checkpoint available	Learned anomaly scoring and localization
Fallback	Automatic	Advanced artifact unavailable/fails	Portable detector keeps inspection available and reports fallback

11.2 Local operation

The local path uses Python 3.13.5, Node.js 20 or later, MongoDB, a root ``.env``, backend requirements, and frontend packages. The frontend defaults to `http://localhost:8000`; Cloudinary and the complete MVTec dataset are optional for website use.

11.3 Cloud deployment

Layer	Service	Configuration responsibility
Frontend	Vercel	Next.js build; `NEXT_PUBLIC_API_BASE_URL` points to backend
Backend	Render	Python 3.13.5; Uvicorn; `/health` service check; one worker
Database	MongoDB Atlas	Connection URI and database name supplied as secrets
Media	Cloudinary	Enabled explicitly with three credentials and storage flag
Models	Repository/runtime volume	Compact artifacts included; advanced checkpoints installed separately when required

Operational note: A free backend instance can cold-start after inactivity. Model warm-up, compact artifacts, and concurrent storage reduce avoidable delay, while hosting capacity still determines peak response time.

12. Practical End-to-End Demonstration

12.1 All-category integration run

Notebook 08 executes the same portable orchestration used by the application on one normal and one defective image from each of the 15 MVTec categories. It processed 30 images with 0 runtime errors. The median measured inference time was 168.4 ms and the 95th percentile was 540.6 ms on the development machine.

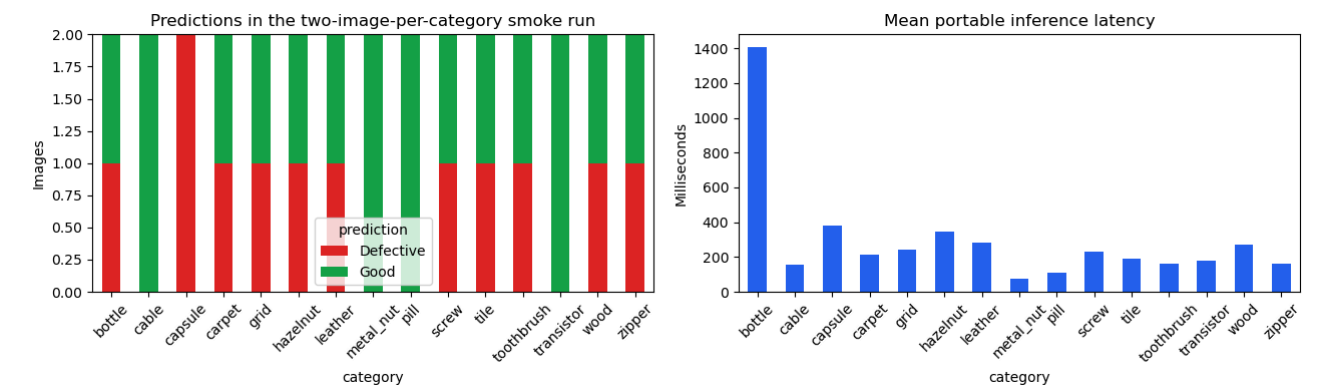


Figure 6. All-category portable inference smoke run
Source: Executed output from `08\_multicategory\_end\_to\_end\_inference.ipynb`.

Interpretation: The 25 matching binary outcomes out of 30 images are a small integration sample, not a formal accuracy estimate. Category metrics and limitations remain in the model evaluation notebook and earlier milestone reports.

12.2 Detailed inspection

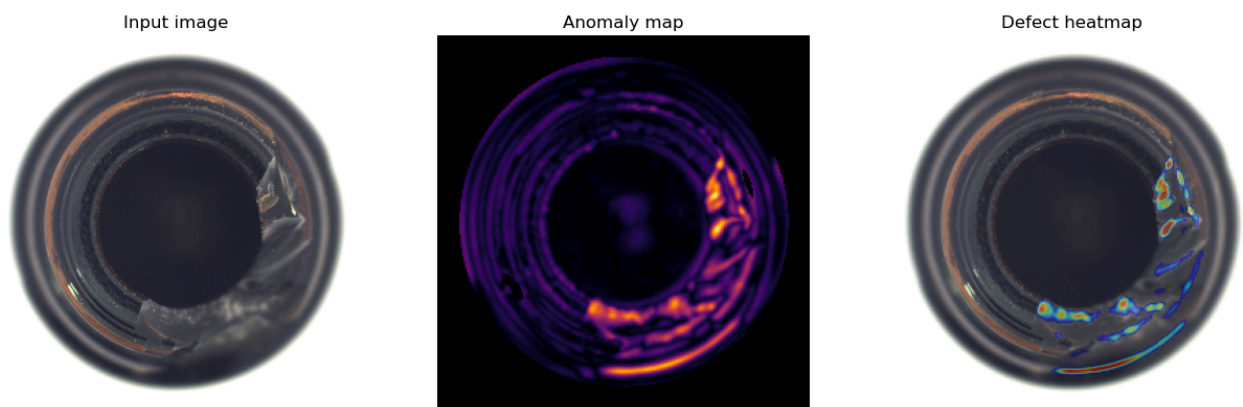


Figure 7. Executed anomaly map and heatmap
Source: Real MVTec bottle image processed by the portable runtime in Notebook 08.

Executed backend-ready inference result

CATEGORY	bottle	DEFECT AREA	3.68%
PREDICTION	Defective	SEVERITY	65.27 / 100
DEFECT TYPE	broken_large	SEVERITY LEVEL	High
DETECTION CONFIDENCE	58.9%	QA DECISION	Fail
SUBTYPE CONFIDENCE	99.7%	RUNTIME	baseline
ANOMALY SCORE	0.047745	FALLBACK	No
Recommended action: Reject product or send to rework based on QA policy			

Figure 8. Executed backend-ready result
Source: Values extracted from the executed notebook output contract.

The representative broken-large bottle result was detected as Defective, classified as broken_large, assigned High severity, and sent to Fail. The result includes separate detection and subtype confidence values, a defect-area estimate, engine name, fallback state, and a recommended quality action.

13. Software Testing and Build Results

13.1 Python test suite

```
PS C:\visioninspect-ai-github> python -m pytest tests -q

..... [100%]

50 passed in 46.55s

Coverage includes API routes, JWT/passwords, upload metadata,
camera samples, 15-category artifacts, preprocessing, anomaly
detection, classifiers, heatmaps, severity, and runtime outputs.
```

Figure 9. Full Python test result
Source: Executed in the full VisionInspect AI repository on 15 August 2026.

Test group	Behavior checked
API and security	Health/live/ready, error structure, OpenAPI routes, JWT, password hashing
Data and preprocessing	Dataset discovery, image shapes/types, grayscale and mask-guided crops
Detection	Thresholds, percentile score, residual cleanup, dual-gate decision
Classification	Artifact loading, ONNX priority, PyTorch-free CNN, compact forest parity
Localization and severity	Heatmap shapes, untinted good images, mask metrics, formula and mappings
Portability	15 category artifacts, SHA-256 descriptors, Render dependencies, no TTL expiry
Application integration	Automatic metadata, storage concurrency, camera samples, backend-ready result

13.2 Frontend production build

```
PS C:\visioninspect-ai-github\frontend> npm run build

Next.js 16.2.10
Compiled successfully in 13.5s
Generated static pages (13/13)

/analytics /camera /dashboard /inspection /login
/model-metrics /reports /rework /upload /users

Production build completed with no compilation error.
```

Figure 10. Frontend production build result
Source: Executed with Next.js 16.2.10 on 15 August 2026.

The build compiled successfully in 13.5 seconds and generated all application routes. This confirms that the delivered frontend source can be compiled for hosting with the current dependency lock file.

14. Operational Limits and Mitigations

Risk or limit	Impact	Mitigation
Factory domain shift	MVTec images do not represent every factory camera and product	Collect target-line data, recalibrate thresholds, and monitor drift
Category quality varies	One model configuration is not equally strong for every category	Use category-specific metrics and promote improved checkpoints selectively
Unknown subtype	Classifier only knows labels present in training data	Return unknown_defect and require manual review
Free hosting latency	Cold start and limited CPU can delay requests	Warm runtime, compact artifacts, request timing, retry message, paid compute when needed
Local media durability	Ephemeral hosting may lose local uploads	Enable Cloudinary or persistent object storage for online operation
Camera scope	Simulation does not prove physical trigger timing	Add vendor SDK/RTSP/PLC integration as a separate factory phase
Safety policy	Severity weights are project rules, not certification	Factory QA owner approves weights, zones, and release thresholds
Advanced artifact size	PaDiM/PatchCore checkpoints are large	Install on compute server or model store; keep compact runtime for portability

14.1 Monitoring points

- Track model engine, fallback status, inference/storage/total time, and request ID per inspection.
- Monitor category-level defect rate, confidence, severity, manual-review frequency, and line trends.
- Inspect false-positive and false-negative examples before changing thresholds.
- Retain reviewer and rework outcomes so model results can be compared with human decisions.

15. Milestone Completion and Project Handover

15.1 Completed deliverables

Deliverable	Completed content
Integrated inference service	Category-aware runtime, storage, timing, explainability, and stable response schema
Operational APIs	Auth, users, inspections, batch/camera, reports, rework, analytics, model, production, audit, health
Persistence layer	Eight MongoDB document types with retained quality history
Frontend workflows	Protected pages for inspection, review, corrective action, reporting, analytics, and administration
Portable model package	15 category profiles/classifiers plus shared ONNX and optional advanced engines
Configuration	Safe environment template, Python pin, Render definition, Vercel definition
Quality checks	50 passing Python tests and successful production frontend build
Documentation	Four sequential milestone reports and executable notebooks

15.2 Handover instructions

Step	Handover action
1	Clone the repository and create a private <code>.env</code> from <code>.env.example</code> .
2	Install backend requirements and frontend packages using the documented versions.
3	Start MongoDB, then start FastAPI and Next.js on the configured ports.
4	Sign in with a development account or request registration approval.
5	Select a category, upload a matching image, and confirm result, heatmap, history, and report behavior.
6	Run the Python tests and frontend production build before changing the model or application code.
7	Install advanced checkpoints only on a suitable compute environment and keep portable fallback enabled.

15.3 Completion statement

Milestone outcome: VisionInspect AI now demonstrates the complete path from image acquisition and AI inference to an auditable manufacturing quality decision. The delivered system is suitable for academic demonstration, team handover, portable local use, and further factory-specific calibration and integration.

References

Reference	Use in this report
Project source	VisionInspect AI full-stack repository and Member 4 AI/ML branch, inspected 15 August 2026
Dataset	MVTec Anomaly Detection dataset: 15 product categories with normal training images, test defects, and masks
Frameworks	FastAPI, Next.js, React, MongoDB/Beanie, OpenCV, NumPy, scikit-learn, ONNX/OpenVINO, Anomalib
Milestone 1	Data Foundation and Portable Baseline - 25 July 2026
Milestone 2	Advanced Anomaly Detection and Localization - 1 August 2026
Milestone 3	Defect Classification, Severity Scoring, and QA Decisions - 8 August 2026

Appendix A. API Inventory

API group	Main routes
Auth	POST /auth/register; POST /auth/login; POST /auth/token; GET /auth/me
Users	GET/POST /users; approve; reject; update; reset password
Inspection	POST inspect; POST batch-inspect; GET camera samples/categories; POST camera simulate; list/get/update/review
Production	GET catalog; POST products; POST lines; POST batches
Rework	List/get/create/update tickets; lookup by inspection
Reports	Create report; authenticated download; list/get reports
Analytics	Filtered summary and authenticated CSV export
Models	Metrics, runtime settings read, runtime settings update
Audit	Filtered audit-log list
Health	Service summary, liveness, and readiness checks

Appendix B. Persistence and Configuration Map

Configuration area	Environment variables
Application	APP_NAME, ENVIRONMENT, SECRET_KEY, ACCESS_TOKEN_EXPIRE_MINUTES
URLs/CORS	FRONTEND_URL, BACKEND_URL, CORS_ORIGINS, NEXT_PUBLIC_API_BASE_URL
Database	MONGODB_URI, MONGODB_DATABASE
Media	USE_CLOUDINARY_STORAGE, Cloudinary credentials, timeout, UPLOAD_DIR
Inference	USE_PADIM_INFERENCE, USE_OPENVINO_INFERENCE, device/accelerator, artifact paths
Performance	OPENCV_NUM_THREADS, WARM_MODEL_ON_STARTUP
Limits	MAX_UPLOAD_SIZE_MB
Development account	BOOTSTRAP_ADMIN_ENABLED plus private account values

Secret handling: `.env.example` contains placeholders and safe local defaults. Real MongoDB, Cloudinary, JWT, and production account credentials must be supplied privately and must not be committed or printed in documentation.

Appendix C. Deliverable and Evidence Index

Evidence	Source	Purpose
Architecture and sequence diagrams	Implemented routes, services, models, and runtime code	System boundaries and data movement
All-category smoke chart	Notebook 08	One normal and one defective image exercised for every category
Heatmap and anomaly map	Notebook 08	Real image localization output
Backend-ready result	Notebook 08 JSON output	Complete scalar contract consumed by the application
Full pytest result	Project test suite	50 passing backend, ML, portability, and integration tests
Frontend build result	Next.js production build	Application pages compile for hosting
Milestone 4 report	This document	Integration, operations, limits, and handover

End of Milestone 4 documentation.